PSAI_097

Polka SignIn

- **Team Name:** Litentry
- **Payment Address:** 0x37a45AdBb275d5d3F8100f4cF16977cd4B0f9Fb7 (USDT)
- **Level:** 1
- **Status:** Terminated

⚠ *The combination of your GitHub account submitting the application and the payment address above will be your unique identifier during the program. Please keep them safe.*

Project Overview

Overview

In a Web2 world, OAuth is the industry-standard protocol for authorization, it obtains an access token–a string denoting a specific scope, lifetime, and other access attributes. Access tokens are issued to third-party clients by an authorization server with the approval of the resource owner. Then the client uses the access token to access the protected resources hosted by the resource server.

In Web3, injected signers are widely used for connecting accounts with dApps, for example, Metamask one-click connect gives the dApp Ethereum accounts information with `eth_account` method, WalletConnect generate a QR code to connect a mobile wallet with its handshake protocol. And after connecting, every user action needs to be signed by the injected Ethereum account, this method is safe and efficient.

Considering injected signers' "connect" feature as a sign-in method is not optimal because a malicious user could inject a custom signer like Metamask with a customized `eth_account` method so that the malicious user could pretend to be the owner of a labeled Ethereum address. So that for an injected signer, the user needs to sign a piece of data for each action, which is not efficient and has a bad user experience.

A combination with OAuth and injected signer should be raised, there is some research and practices in the industry as listed below:

EAuth Sign in with Magic links rely on JsonRpcProvider DAuth registration by decentralized authorities. ethereum oauth This paper [OAuth 2.0 authorization using blockchain-based tokens] auth0 team has make a proposal

to ask the user to sign such a magic string with injected signers and provides it as an access tokens for Applications. Instead of verified by authorization server who holds the approval of resource owner (user), it should be verified in a decentralized way.

Project Details

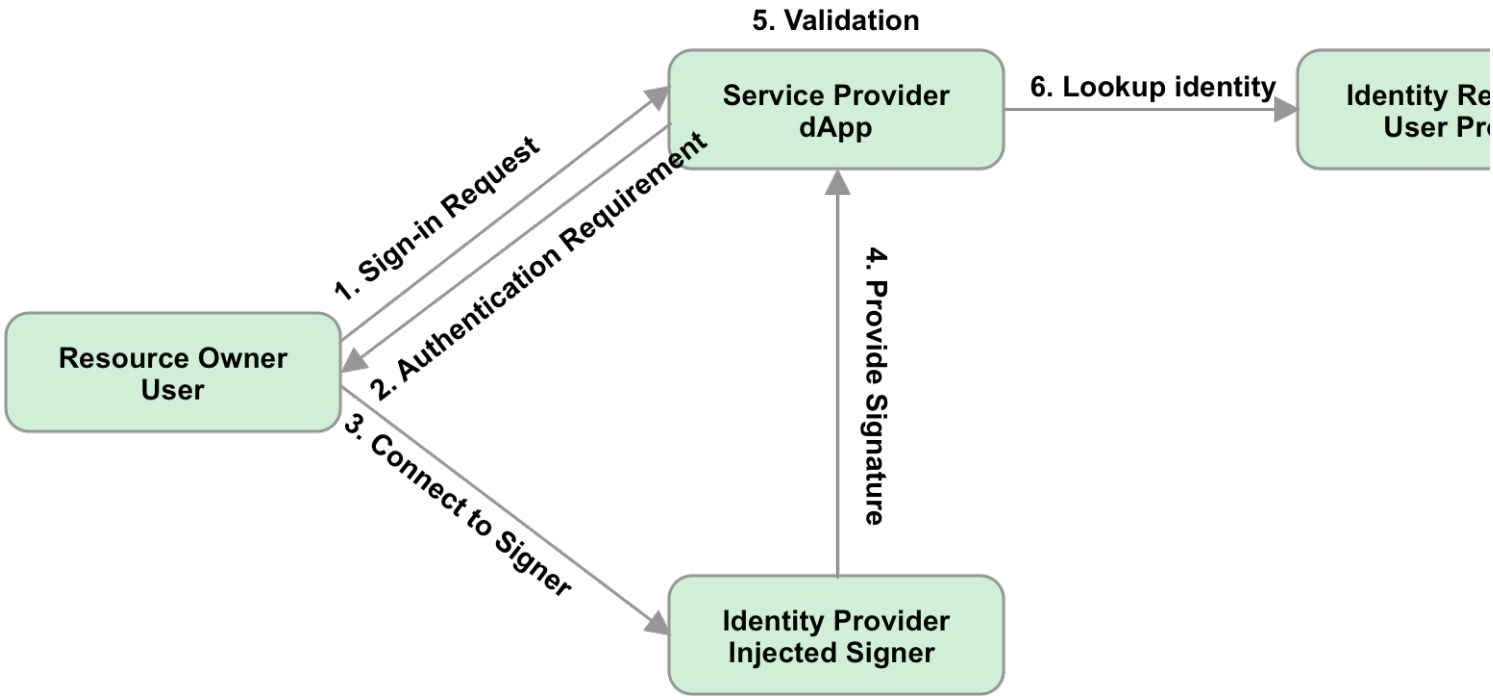
Sign in Participants

- **Service providers** are the dApps that have the sign-in requirement.
- **Identity providers** are the injected signers which provide the authentication information of the user who holds the wallet.
- **Resource Owners** are the users who hold the self-custody crypto wallets.

Workflow

The workflow works the same with or without OAuth specification. Only the 4th step will differ.

1. A user sends a request to a service provider.
2. The service provider sends a callback address, the sign-in challenge, and required permissions to the user.
3. The user connects to the injected signer.
4. injected signed to sign the challenge, and send the signature together with the public address, an access token specifying the scopes like expire time, back to the callback address. With OAuth Implementation, the access token is compliant with the OAuth standard.
5. The service provider validates the signature, if successful, the user signed in,
6. If there is no related user information in the database, the service provider will lookup for the user information from Identity Registrar, if related TEXT records are found for this address, the information will be imported into the service provider's database.



Sign-in Flow

Implementation

1. Sign-in Request

   User want to sign in the dApp by send a simple request with identity chain type, eg: DOT, KSM, ETH etc.

   The payload is required since each chain has its own algorithmic mechanism.

   request payload(1)

   ```
   {
       "identity-type": "dot"
   }
   ```

2. Authentication Requirement

   dApp will return the callback info. The callback endpoint is used by Identity provider to send signature data to dApp.

   ```
   {
       "identity-type": "dot",
       "callback-endpoint": "http://dapp.com/login/callback",
       "scope": ["xxx", "yyy"],
       "chanllenge": xxxxxxxxxx",
   }
   ```

3. Connect to Signer

   User sends request to the Identity Provider (Injected Signer like : Polkadot.js Extension, Metamask, Parity Signer ...).

   This action will call up a browser extension or some other external applications.

   {

```
    "identity-type": "dot",
    "callback-endpoint": "http://dapp.com/login/callback",
    "scope": ["xxx", "yyy"],
    "chanllenge": xxxxxxxxxx",
}
```

4. Provide Signature The Identity Provider will generate the Signature data.

```
{
    "identity-type": "dot",
    "public-key": "xxxxxxxx",
    "account-signed": "xxxxxxxxxx",
    "scope": ["xxx", "yyy"],
}
```

explains:

- identity-type

  This field is used to choose the chain type and account.

- public-key

  With the chain account chosen, get the public key of the account. This field will be used by dApp to decrypt the data and verify data consistency.

- account-signed

  use private key to sign the account.

  eg:

```
account = 0x1016f75c54c607f082ae6b0881fac0abeda21781

private-key = 18e14a7b6a307f426a94f8114701e7c8e774e7f9a47e2c2035db29a206321725

account-signed = ECIES ( ${account} , ${private-key})

# encrypt account with private-key by the specified algorithmic mechanism: ECIES
```

- scope

  scope define the permission needed for the dapp to interact with the account.

- chanllenge

  a text string need to be signed by the private key.

The Identity Provider will send the signature data to the callback endpoint of dApp by step #3.

5. Validation

The dApp receives the signature data and do the verification.

```
 ```
 {
     "identity-type": "dot",
     "public-key": "xxxxxxxx",
     "account-signed": "xxxxxxxxxx",
     "scope": ["xxx", "yyy"],
 }
 ```
```

Verification Steps:

- use **public-key** to decrypt the data of **account-signed** , this progress should be successful, and get the account address.

  public-key pairs with **private-key**, this step proves the validity of the private key, and ensure that the data was not tampered with.

- use **public-key** to generate the address by the specified algorithmic mechanism according to the chain type, and get the account address refer to **public-key** .

- verify the two account address , success if they are the same.

- if failure, it means the public key does not match the account address, it may happen when some malicious users want to impersonate an account.

- if success, the dApp should return the response to the Identity Provider with the payload:

```
{
    "verified": true,
    "access-token": "xxxxxx"  # generated by dApp as access token
}
```

6. Lookup Identity

The dApp gets the account address . It can retrieve the related information of account from the external service providers , such as ENS, Polkadot/Kusama Identity Registrar, etc.

Ecosystem Fit

There is few solution combine the OAuth and self-custody wallet, and no such solutions in the Polkadot ecosystem. With our solution and the ecosystem tools like polkadot.js extension and Parity Signer, the substrate account could be used to sign in any web2 or web3 platform which support OAuth, user also do not need to input all his information once registered in a new platform, the information could automatically be fetched from the info records in `identity` pallet, which could gain huge adoption of Substrate account in Web2 world.

Team :busts_in_silhouette:

Team members

- Hanwen Cheng
- Yunjian Bian
- Eric Zhang

Contact

- **Contact Name:** Hanwen Cheng
- **Contact Email:** hanwen@litentry.com
- **Website:** www.litentry.com

Legal Structure

- **Registered Address:** Eichhornstr. 3, 10785 Berlin.
- **Registered Legal Entity:** Litentry Technologies GmbH.

Team’s experience

Litentry Technologies GmbH is a Berlin-based technology company, the team builds the identity-related infrastructure of Web3, builds a Decentralized Identity Aggregation protocol across multiple networks, it features a DID indexing mechanism, and a Substrate-based credit computation network. The protocol provides a decentralized, privacy-preserving interoperable identity aggregation service that mitigates the difficulty of resolving agnostic DID mechanisms. The team has lots of experience in the DID field and has a strong background in Web3 technology. Current products include Litentry Network, My Crypto Profile, and a Governance-focused mobile App.

Team Code Repos

- https://github.com/litentry
- https://github.com/litentry/litentry-parachain
- https://github.com/litentry/litentry-app

Github accounts:

- https://github.com/hanwencheng
- https://github.com/bianyunjian
- https://github.com/jingleizhang

Development Status :open_book:

The development is not started yet.

Development Roadmap :nut_and_bolt:

As the project is small, we only have 1 milestone to be finished.

Number	Deliverable	Specification
0a.	License	Apache 2.0 / GPLv3 / MIT / Unlicense
0b.	Documentation	We will provide both **inline documentation** of the code and a basic **tutorial** that explains how a user can (for example) spin up one of our Substrate nodes and send test transactions, which will show how the new functionality works.
0c.	Testing Guide	Core functions will be fully covered by unit tests to ensure functionality and robustness. In the guide, we will describe how to run these tests.
0d.	Docker	We will provide a Dockerfile(s) that can be used to test all the functionality delivered with this milestone.
1a.	Research	Survey and discuss with related team 20 FTE hours
1b.	Standard Development	Create the specification w/o OAuth 20 FTE hours
1c.	Create Library	Create a Javascript/Typescript library 50 FTE hours
1d.	Real Use Case	Integration with a dApp as first use case 30 FTE hours

The hourly pay rate is 50 USD/hour

In total is 120 hours, and the total payment will be 6000 USD equivalent USDT.

Future Plans

We planned to have a web app for user to manage the access control of the logged platforms, and enable user to curate the profile information in the future.

It would be great to integrate with popular auth solutions in web technology, like passport (node js), next-auth (next.js).